Relatório de Entrega — TaskAnalyzer (Fase 2)

Bootcamp III — Projeto `sdd-task-analyzer`

Data da execução registrada: 17/10/2026

1. Objetivo da Fase 2

Implementar a função `analyze_tasks` e a exceção `TaskValidationError`. A função calcula métricas de produtividade de um conjunto de tarefas: totais, tempo médio de conclusão (geral e por prioridade) e taxa de atraso. Ela também valida a consistência das datas.

O trabalho foi conduzido por especificação (SDD) e seguiu regras de governança explícitas.

2. Escopo executado

- Especificação formal do contrato em `specs/task_analyzer_spec.md`.
- Regras de governança em `CONTEXT_RULES.md`.
- Implementação em `src/task_analyzer.py`, usando apenas a biblioteca padrão.
- Suíte de 62 testes automatizados em `tests/test_harness.py`.
- Exemplo executável em `examples/demo.py`.
- Documentação: README, matriz de rastreabilidade, checklist e roteiro do vídeo.
- Histórico Git local organizado em branches.

Ficou **fora do escopo executado** a publicação no GitHub e a abertura do Pull Request. O motivo foi a falta de autenticação do GitHub CLI no ambiente (seção 9).

3. Fontes e requisitos

Fontes consultadas:

- as regras de governança;
- a especificação de negócio fornecida pelo autor;
- o prompt mestre da Fase 2.

O **enunciado oficial, o barema e os artefatos da Fase 1 não estavam disponíveis**. Por isso, as lacunas foram resolvidas por decisões explícitas (seção 5).

Requisitos implementados:

Grupo	IDs
Funcionais	RF-01 a RF-11
Não funcionais (Context Rules)	RNF-01 a RNF-05
Regras de negócio	RN-01 a RN-08
Exceções	EX-01 a EX-08
Casos de borda	CB-01 a CB-08

O detalhamento de cada requisito está em `docs/matriz_rastreabilidade.md`.

4. Arquitetura e organização do código

O módulo `src/task_analyzer.py` está dividido em responsabilidades pequenas:

Componente	Responsabilidade
`TaskValidationError`	Exceção de domínio (subclasse de `ValueError`, com `task_id`).
`Task`	Dataclass imutável que representa uma tarefa validada.
`AnalysisResult`	`TypedDict` que descreve o contrato de saída.
`_collect_raw_tasks`	Aceita a coleção de entrada e rejeita tipos inválidos.

`_parse_task` e validadores de data	Validam campos e datas e normalizam a prioridade.
`_is_late`	Aplica a regra de atraso (RN-04).
`_safe_mean` e `_safe_percentage`	Fazem os cálculos com proteção contra divisão por zero.
`analyze_tasks`	Orquestra a análise: valida tudo primeiro e só depois calcula.

A função pública não tem efeitos colaterais e não altera a entrada.

5. Decisões técnicas e regras de negócio

As decisões abaixo cobrem pontos que as fontes não definiam. Todas estão registradas na especificação (seção 8).

1. **Datas apenas como `datetime`.** Textos não são convertidos automaticamente; uma data inválida gera `TaskValidationError`.
2. **Definição de atraso.** Uma tarefa está atrasada quando foi concluída depois do `prazo`, ou quando está pendente e a `referencia` já passou do `prazo`.
3. **Taxa de atraso.** É calculada sobre `total_tarefas`, para que pendentes atrasadas também sejam contadas.
4. **Parâmetro `referencia`.** Torna o atraso de tarefas pendentes determinístico e testável.
5. **Média por prioridade em chave própria.** A chave é `tempo_medio_conclusao_horas_por_prioridade`. Assim, `tempo_medio_conclusao_horas` continua sendo um número com o nome exigido.
6. **Arredondamento.** Os valores decimais são arredondados para 2 casas.
7. **Prioridade normalizada.** É aplicado `strip().lower()`, de modo que “Alta” e “alta” formam o mesmo grupo.

6. Estratégia de testes e resultados

Estratégia adotada:

- Os valores esperados foram calculados **manualmente**; o conjunto de referência está na seção 10 da especificação.
- Os testes estão organizados em cinco grupos: sucesso, taxa de atraso, casos de borda, erros e governança.
- A governança é verificada por introspecção: o teste confere type hints e docstrings em todas as funções.
- Todos os testes que dependem de tempo recebem `referencia` explícita, para serem reproduzíveis.

Resultados **reais** da execução local em 17/10/2026:

Ambiente	Comando	Resultado
Python 3.13.16, pytest 9.1.1	`pytest --cov=src --cov-branch --cov-report=term-missing`	**62 aprovados, 0 reprovados**
Python 3.11.17	`pytest -q`	**62 aprovados, 0 reprovados**

Cobertura medida com pytest-cov 7.1.0:

Arquivo	Linhas	Ramos	Cobertura
`src/task_analyzer.py`	112	38	100%

7. Evidências de versionamento

Repositório Git local com três branches:

Branch	Função
`main`	Versão estável. Contém apenas a estrutura inicial até o merge do Pull Request.
`develop`	Integração. Criada a partir de `main`.
`feature/task-analyzer-phase-2`	Desenvolvimento da Fase 2.

Commits na branch `feature/task-analyzer-phase-2`, em ordem cronológica:

1. `chore: initialize project structure`
2. `docs: define task analyzer specification`
3. `feat: implement task analysis metrics`
4. `test: cover task analyzer requirements`

5. `docs: add delivery report and evaluation checklist`

Para conferir o histórico, use `git log --oneline --graph --all`.

8. Links

Item	Situação
Repositório (`kaykyferreira7/sdd-task-analyzer`)	**Não publicado.** Ainda não confirmado.
Pull Request	**Não criado.** Ainda não confirmado.

9. Limitações e pendências

1. **Publicação no GitHub.** No ambiente de desenvolvimento, `gh auth status` informou que o token configurado era inválido. Por isso, não foi possível:
   - verificar se o repositório já existia;
   - criar o repositório;
   - publicar as branches;
   - abrir o Pull Request.
2. **Autoria dos commits.** Os commits locais foram registrados com a identidade padrão do ambiente de desenvolvimento. Antes de publicar, o aluno deve configurar seu nome e e-mail e redefinir o autor:

   ```
   git config user.name "Seu Nome"
   git config user.email "seu-email@exemplo.com"
   git checkout feature/task-analyzer-phase-2
   git rebase --root --exec "git commit --amend --no-edit --reset-author"
   RAIZ=$(git rev-list --max-parents=0 HEAD)
   git branch -f develop "$RAIZ"
   git branch -f main "$RAIZ"
   ```

   Isso só deve ser feito **antes** da primeira publicação, porque reescreve o histórico local.
3. **Enunciado e barema oficiais.** Não foram consultados, porque não estavam disponíveis. Se houver divergência, a especificação, o código e os testes devem ser ajustados.
4. **Vídeo.** O roteiro está pronto em `docs/roteiro_video.md`; a gravação e o envio cabem ao aluno.

10. Conclusão

A Fase 2 está implementada, especificada e testada localmente:

- 6 métricas calculadas;
- validação completa das entradas;
- proteção contra divisão por zero;
- 62 testes aprovados e cobertura de 100%.

A entrega fica completa depois de três passos: publicar o repositório, abrir o Pull Request e gravar o vídeo, conforme a seção 9.